Overview

Semantic Governance Lifecycle & Services Overview

This document is a more detailed breakdown of the Semantic Services we need to stand-up as part of our MVP1 implementation. There are details covered in a separate document related to specific functionality pertaining to our MVP1 Prep. This is also not covering the work we are pursuing to setup Semantic Governance. There are outstanding dependencies that Sherri and I will identify with Metaphacts to support our processes for version control, change management, quality management, and publishing.

The more detailed diagrams covering the Semantic Services can be found in slides 12-14 in the Semantic Governance walk-around deck

Semantic Services (Inbound)

Within the scope of MVP1, we will be standing up a manually triggered (headless) process for loading data structures into Metaphactory (Inbound). List of structures and processes are outlined in the 2nd tab.

Semantic Services (Processing)

This will be a more robust set of features we need to have in place to demonstrate mapping at scale and extending knowledge. There are a number of dependencies identified in the MVP1 Prep doc (referenced above). The processes are outlined in a stepwise manner in the 3rd tab.

Semantic Services (Outbound)

The outbound will leverage the OOTB capabilities in Metaphactory leveraging their AI Prompt interface due to limited time and resources. Future capabilities will focus on integration with 3rd party AI prompts and VZ systems (notably Marketplace and Catalog). In addition, the lessons learned from the integration with the VZ Gemini Vector DB as part of the Metaphactory Metis AI framework should provide a template for integrations with Vector DBs for other VZ AI Models.

Test Planning

Top-Down Artifact V&V

- **Verification:**
  - **Logical Consistency**
    - Check for contradictions, disjoint classes, or logical inconsistencies introduced by the change.
  - **Syntax Validation**
    - Check the artifact's syntax against its language specification (SPARQL endpoint for RDF, SHACL validator for constraints).
  - **Standards Compliance**
    - Check that new classes or properties adhere to established naming standards.
    - Check that taxonomies are tied back to vocabularies
    - Ensure there are no orphan classes
    - Review metadata and version management
- **Validation:**
  - **Stakeholder Review**
    - Where possible engage SMEs to review changes for relevancy and accuracy within the domain.
  - **Use-Case Scenario Testing**
    - Build a “regression” test bed of scenarios to ensure consistency, relevancy, and accuracy.
    - Generate prompt-based questions
    - Create SPARQL queries about each artifact
    - Create persona and domain-specific queries to evaluate relevancy

Bottom-Up Artifact V&V

- **Verification:**
  - **Schema Conformance**
    - Check the artifact's syntax against its language specification (SPARQL endpoint for RDF)
    - Check the ontological representation of a digital asset's schema correctly conforms to the structure of the source data (including RI)
  - **Data Type & Format Checks**
    - Verify that data types (e.g., `xsd:date`, `xsd:integer`) are correctly applied and that value formats are preserved.
- **Validation:**
  - **Completeness Check**
    - Validate that all necessary parts of the digital asset's schema have been correctly modeled (project id, database name, table name, columns)
    - **Query Test**

        - Create sample SPARQL queries against the represented schema to confirm it can generate a proper SQL query (assuming relational structure in this case) to ensure it can be used for downstream orchestration (data retrieval and analysis).

Mapping Ontologies V&V

- **Verification:**
    - **Mapping Rule Syntax**
        - Verify the relations stem from the Upper Ontology (BFO)
        - Verify that the mapping rules are syntactically correct and logically sound
    - **Identity & Equivalence**
        - Verify that the relationships are logically consistent with both source (Top-Down) and target (Bottom-Up) ontologies
        - Note: Source & Target in some cases will both be Top-Down (e.g. Mapping VZ Knowledge with 3rd Party Knowledge (TMForum, ISO, Published / Certified Taxonomies and Ontologies)
- **Validation:**
    - **Sample Data Transformation**
        - Run a small, representative data set through the mapping and manually inspect the output to confirm that data is transformed accurately and completely.
    - **Coverage Analysis**
        - Verify that all required elements from the source schema have a corresponding mapping to the target ontology
        - Verify the number of relations from Top-Down to Bottom-Up

Integration Testing

- **Semantic Services (Inbound):**
    - **Intake Pipeline:**
    - **Ingestion Pipeline**:
- **Semantic Services (Processing):**
    - **Integration with AI**:
        - Configuration
        - Training
        - Dashboard
- **Semantic Services (Outbound):**
    - **Push**:
        - Publishing Test Cases

- **Pull:**
  - Downstream Semantic Search (using Metaphactory APIs)

System Testing

- **System Fixes:**
  - Test Cases
- **System Upgrades:**
  - Test Cases
- **System Customizations:**
  - Test Cases

Release Management

System

- **Security:**
- **Templates:**
- **Upgrades:**
- **AI Training & Learning:**
- **Services (Inbound, Processing, Outbound):**

Top-Down Artifacts

- **Vocabularies:**
  - **Standards Compliance**
    - Core
    - Domain-Specific
- **Taxonomies:**
  - **Standards Compliance**
    - Core
    - Domain-Specific
- **Ontologies:**
  - **Standards Compliance**
    - Core
    - Domain-Specific

Bottom-Up Artifacts

- **Ontologies:**
  - **Standards Compliance**
    - Ingestion Pipeline
    - Conversion & Fidelity

Mapping Ontologies

- **Standards Compliance**
  - Core
  - Domain-Specific

Semantic Services (Inbound)

Semantic Services (Inbound).

INBOUND Flow

Not sure if we need all of these steps for auth and provisioning as exists in MDI for the Catalog

We can vet this out with David and team as we finalize the Arch and Systems Design

- Request
- Scheduler
- Search for Digital Asset
    - Search Catalog
- Extract Digital Asset
    - Obtain URL/URI
    - Obtain Access Point
    - Obtain Authorization
    - Obtain Credentials
    - Obtain Schema, Version, & Datetime Stamp
    - Obtain Attributes (list)
    - Assemble Inputs
    - Generate RDF Structure
    - Add Metadata for Ontology Template
    - Validate RDF Structure
    - Create .TRIG file
- Load Digital Asset into Metaphacts
    - Check if Ontology exists
        - If NO, set Ontology version to 1.0
        - If YES, Increment Ontology version
    - Load Ontology into Metaphactory
    - Validate Ontology (SPARQL Query)
    - Close out Request
    - Notify SG Operations
    - Update SG Workflow
        - Inbound Queue

Architecture Overview

This framework merges microservices, agentic orchestration, and semantic governance automation, producing a repeatable and scalable inbound data handling model that conforms to MBSE-aligned digital engineering architectures.

The goal is to build a robust automation architecture for Inbound Processing in Semantic Governance that can be built using a network of microservices, coordinated AI Agents for decision-driven orchestration, and semantic tools like RDFLib integrated with Metaphactory APIs. Each service handles one atomic function for clear traceability and schema compliance, while agents use orchestration logic

This pipeline aligns with semantic governance principles, combining AI orchestration, microservice modularity, and knowledge graph operations.

Semantic Services (Inbound).

Core components include:

1. Microservices Layer (Python REST Services) for discrete operations.
2. AI Agent Layer using Semantic Kernel or LangChain for sequencing, reasoning, and error resolution.
3. Semantic Data Layer using RDFLib for RDF modeling, and Metaphacts Graph Store API for ingestion and validation.

Workflow

1. Inbound request enters queue and invokes pipeline.
2. The AI orchestrator calls microservices per order.
3. RDF file generation (.TRIG) occurs post validation.
4. File uploaded to Metaphacts Graph Store; ontology version validated.
5. Workflow entry updated; SG notified.

AI Agents and Orchestration Logic

Using Semantic Kernel's multi-agent orchestration framework, three main AI agents can manage this pipeline:

- Orchestration Agent: Central decision unit that invokes sequential microservices (Sequential Pattern).
- Validation Agent: Evaluates ontology and SHACL constraint compliance.
- Governance Agent: Audits metadata and ensures policy conformance for schema naming and version control.

Each agent runs under Semantic Kernel orchestration, utilizing Sequential and Handoff patterns:

- Scheduler → Asset Retrieval → RDF Generation → Validation → Loading → Ontology Versioning → Notification.
- Agents share context stores to ensure schema, metadata, and governance coherence.

Semantic Governance Compliance

Governance rules align roles, privileges, and review checkpoints:

- Policy Framework: Defines approval thresholds for ontology promotion.
- Metadata Governance: Ensures proper schema linkage and provenance attributes.
- Lifecycle Tracking: Captures versioning and timestamps in RDF metadata.
- Audit Trail: Updates Semantic Governance (SG) workflow with activity logs.

Semantic Services (Inbound).

Microservices Breakdown

Service	Function	Tools / Libraries
Request Scheduler	Queue inbound requests, assign IDs, and trigger workflow.	Celery + Redis
Search Digital Asset	Query asset repository with metadata filters.	ElasticSearch / Custom API
Search Catalog	Locate relevant schema or ontology source.	Metaphacts Query Catalog API
Extract Digital Asset	Download or parse content from stored archive.	Python requests, JSON / XML parser
Obtain URL/URI & Access Point	Resolve persistent identifiers.	W3C URIRef via RDFLib
Obtain Authorization & Credentials	Token exchange with OIDC or API keys.	Authlib / Keycloak
Obtain Schema, Version, Datetime	Retrieve ontology meta-data and constraints.	SPARQL Query via RDFLib

Semantic Services (Inbound).

MVP1 Scope

Ingestion Pipeline

- Goal: to convert the list of Data Structures to RDF
- Need to import 12 Data Assets
    - Starting with 2 Data Products
        - RAN
        - Interactions (Omni)
    - Here is an additional list of priority data sources that Mahesh culled from the Slack Channel (vz-data-discovery):
        - Credit_app_v
        - Cust_acct_line_v
        - Cust_acct_svc_prod_trans_v
        - dla_sum_fact_v
        - Equip_sum_fact_v
        - Equipment_item_v
        - Icm_summary_fact_v
        - Payment_v
        - Pos_trans_v
        - Rev_sum_fact_bl_v
    - Most of the tables are from Teradata, but they should be loaded in the Catalog so we don't need a GCP Project ID. Some of these tables are used in many GCP Projects.

Load the RDF files as class objects into Metaphactory as Bottom-Up ontologies

- Part of the hardening of the pipeline includes bringing in additional information from the Catalog for the tables and columns
    - Attributes (properties)
    - Descriptions (properties)

Stretch Goals

- Eventually, we can load Metrics
    - attributes (as properties)
    - description (as properties)
- Eventually, we can load Hierarchies
    - attributes (as properties)
    - description (as properties)
- We should also see if we can load in LookML Projects
    - So we can map to Data Assets, Metric, Data Products, Hierarchies

So, getting our pipeline up and running is a priority for ~~June~~ August so we can prep so we can start ramping up our Mapping at Scale

Semantic Services (Processing)

Semantic Services (Processing)

Top-Down Modeling (Setup)

We need to review the list of Bottom-Up Schemas and identify any gaps in our Top-Down BEFORE we begin mapping. Key actions include:

- Create new ontologies, taxonomies, and vocabularies
- Extend existing ontologies, taxonomies, and vocabularies

- Initial scope for domain-specific ontologies involves: (partial)

    - Network > Wireless
        - Assets
            - Equipment
    - Sales
        - Account
        - Customer
        - Retail
            - POS
    - Finance
        - Procurement
        - Revenue
    - Legal
        - Contract
    - Supply Chain
        - Equipment

- There are 2 options for setting up the Top-Down
    - Option 1 - the KE team (within Semantic Governance) does this manually
    - Option 2 - the Metis AI tool suggests updates via the AI Dashboard
        - The KE team will review and approve the updates
        - The Metis AI tool applies the updates to extend knowledge

Mapping Ontologies

- Create separate mapping ontologies for each Bottom-Up schema
    - Naming Conventions

Semantic Services (Processing).

- Reverse Engineer the list of relevant Top-Down DS Ontologies
  - Note: This is a recursive process with Top-Down Modeling
- Build the Mapping Ontology

Mapping at Scale

- Run the Mapping at Scale
  - Generate relations between classes in the Mapping Ontology
    - Directionality Top-Down to Bottom-Up
    - Cardinality Top-Down M:1 Bottom-Up
  - Apply naming conventions for relations using the Upper Ontology (BFO)
    - Note: we may need to expand the list as we step through mapping
  - Apply confidence levels
- Train the Mapping
  - KE to review the recommendations
    - Approve suggestions
    - Refuse suggestions
      - Provide reasons (this is critical)
        - We may have more than one reason
        - How we capture each reason as a statement is critical to support negative learning (need to discuss with Metaphacts team)
          - May need to extend the AI Dashboard
    - Complete the suggestions for each Mapping Ontology
- Rerun the Mapping at Scale
  - Approve
  - Refuse
  - Assess Learning and Improve Training Feedback
  - Continue with reruns until the AI learns and achieves 100%
    - Need to define “Good Enough” and identify gaps
    - We may only need 1 or a few relations per schema element
  - Assess cycle time for each Bottom-Up schema
    - Ideally, this will start to go down as we encounter more elements
    - Similarities, nearest neighbor, Bert, etc. should improve the accuracy

Note: For simple tables / views, there is only 1 table and a list of columns. But, for databases and data products, there may be several sets of tables and we need to preserve / incorporate RI

Semantic Services (Outbound)

Semantic Services (Outbound).

In order to demonstrate Semantic Search, we need to focus on the business friendly ability for any persona to search, discover, and find specific types of data (columns and locations). We are not planning on loading sample data into Metaphactory since that is not the direction for the company. For the purpose of this demonstration, we will leverage the OOTB UI from Metaphactory to support NLQ (natural language query) to SPARQL translation. This supports prompt engineering efforts and should demonstrate relevancy (aligning domains, concepts, and content) in a seamless navigation from HL enterprise views to specific details.

Since we have built the Top-Down using a component design and are designing separate mapping ontologies for each Bottom-Up structure (ingested in our Inbound service), we need to showcase how federated graph queries work in conjunction with the above mentioned relevancy. The following 5 areas are outlined below for the demo

NLP Query

- Natural Language Query (NLQ) to SPARQL Translation
- Federated Query Execution
- Inferencing in Action (not sure we can show this since we did not enable this in our install for Metaphactory and we have not defined any inferencing rules). Need to discuss with vendor

Visualization & Explanation

- Present graph query results in charts, graphs, KGs, dashboards for users to intuitively explore and understand the graph
- Explanation generation showing how the system generates natural language explanations for query results and contextual insights based on persona (TBD)

Highlighted Benefits

- Enhanced relevancy (contextually aware AI)
- Improved accuracy (reduced hallucinations in AI)
- Faster Performance (vector similarity should accelerate search)
- Dynamic *Agent* AI Reasoning (Agentic AI leverages current domain knowledge)
- Unified View (aggregates, integration, and organization of data into unified views should simplify search while reducing ambiguity for personas)
- Interoperability (span silos of data without programmatic intervention)
- Auditability (provenance tracking showing how AI decisions map to ontologies)
    - 🌕 Note: this would require publishing into Vector DB to support the AI prompt (TBD)

Semantic Services (Outbound).

Continuous Learning

- Recursive Learning (leveraging positive & negative feedback)
- Demonstrate how AI actions and outcomes feed back into the SKL
    - Mapping Ontologies (Top-Down to Bottom-Up)
    - Knowledge Extension (Top-Down enhancements to Domains, Sub-Domains, Taxonomies, and Vocabularies) to refine relationships and improve accuracy
- Scoring (confidence scores) added to relationships to guide path optimization and improve explanation

Note: There is a separate design specification for Semantic Services (Outbound) that is meant to support the integration and handoffs with a 3rd party system (aka Marketplace) to facilitate NLP handoffs covering a subset of Search scenarios.

Semantic Services (Outbound)

Search (Semantic & Conversational)

1. Query Types

a. Simple Queries

- Purpose: Provide quick access to basic information.
- Inputs:
  - Parameters (key-value pairs)
  - Terms (list of keywords)
  - Acronyms (list of abbreviations)
  - Synonyms (term-to-synonym mappings)
- Outputs:
  - Results matching the parameters and text-based filters.

b. Advanced Queries

- Purpose: Answer more detailed domain-specific questions.
- Inputs:
  - Natural language phrases or questions.
  - Domain specification (e.g., network, finance).
- Outputs:
  - Results enriched with contextual domain-specific information.

c. Complex Queries

- Purpose: Solve multi-faceted queries involving digital assets across domains.
- Inputs:
  - Phrases and natural language questions.
  - Constraints (data, data products, metrics, AI models, etc.).
  - Single or multiple domains.
- Outputs:
  - Contextual insights with federated data from multiple domains.

2. Federated Graph Queries

- Purpose: Enable querying across domain-specific ontologies.

- Features:
    - SPARQL for querying federated graphs.
    - SHACL for constraint validation and semantic reasoning.
    - Mapping terms and synonyms to domain-specific ontologies.

3. Core Capabilities

- Search: Efficiently retrieve relevant knowledge base items.
- Contextual Mapping: Link terms and queries to ontological concepts.
- Relevancy Scoring: Prioritize results based on context and query constraints.
- Prompt Engineering: Return query-specific results for human and machine-generated prompts.

Architecture

1. Components

- Query Builder:
    - Constructs SPARQL queries based on input types (simple, advanced, complex).
    - Leverages domain-specific ontologies and SHACL constraints.
- Execution Engine:
    - Executes SPARQL queries against the knowledge base or federated endpoints.
    - Integrates results from multiple data sources.
- Ontology Mapper:
    - Maps terms, synonyms, and acronyms to ontology entities.
    - Ensures cross-domain consistency.
- Relevance Engine:
    - Scores results based on query constraints and context.
    - Supports ranking for both human-readable and machine-interpretable outputs.

2. Query Flow

1. Input Parsing:
    - Identify query type and parse inputs (e.g., terms, constraints, domain).

Semantic Services (Outbound).

2. Ontology Mapping:
    - Map input terms to entities in domain-specific ontologies.
3. SPARQL Query Generation:
    - Generate queries for the given type (simple, advanced, or complex).
    - For complex queries, generate federated queries across domains.
4. SHACL Validation:
    - Validate results against SHACL constraints for semantic accuracy.
5. Execution and Results:
    - Execute queries on the knowledge base or federated graph.
    - Rank and return results based on relevancy scoring.

3. Federated Query Design

- Federation Clause:
    - Use SERVICE keyword in SPARQL for querying across domains.
- Domain-Specific Constraints:
    - Apply SHACL shapes for validation.

Non-Functional Requirements

- Scalability: Handle large datasets and federated queries efficiently.
- Performance: Optimize query execution and result retrieval.
- Extensibility: Support future ontologies and query types.
- Security: Ensure secure access to federated endpoints and sensitive data.

Key Challenges:

- First, we need to outline a working service facade to capture requests of any type (simple token-based, boolean, phrases, and eventually questions)
- Second, we need to reduce the complexity associated with navigating multiple KGs (ontologies) to encourage adoption and use of Semantic Knowledge. These require federated Graph Queries across multiple ontologies. This needs to be abstracted from the consumer

Semantic Services (Outbound).

- Need to Identify WHO using Persona and mapping this to our Unified Persona Model to support Relevancy (for now need to mockup a working example in Metaphactory and build out the basic content in Postgres using the DLL provided by Colin B).
- Need to clarify the WHAT using Semantic Preprocessor for the Request in conjunction with a GraphLLM we can map the context associated with the Request (Inference the Domains associated with Request and Relevant with the Persona)
- Need to clarify the WHERE & WHEN when applicable by mapping elements from the Request to Location and Time ontologies
- Need to clarify the WHY by mapping elements from the Request to Events across one or more Process ontologies within each Domain

- Third, depending on the type of consumer we may need to trigger one or more Agents for additional downstream processing. Invocation of these agents will depend on integration and orchestration capabilities within the 1VZ Data ecosystem (still evolving).
  - Data Services - extract data from one or more sources
  - Analytic Services - apply calcs and hierarchies to data extracts to generate measures, analytics, and/or metrics
  - Visualization Services - apply visualizations to simplify consumption and improve understanding of the analytic output
  - Intelligent Services - AI flows (for now) and AI Agents (future) to generate responses and provide explanations about known and generalized information about related unknowns covering current events and future events (projections).